

SOFTWARE ENGINEERING DOCUMENTATION

TECHNICAL ARCHITECTURE DOCUMENT

KYC Verification System

A Full-Stack POC Project to be built using React.js, Spring Boot, and MySQL

Document Title	KYC Verification System
Document Version	1.0
Document Status	Final
Prepared By	Jenisha J
Date	July 2026

1. Introduction

A KYC (Know Your Customer) Verification System is a software application that helps a bank or financial institution to confirm the identity of a new customer before allowing them to open an account or use a service. Instead of a staff member checking paper documents by hand, the system lets a customer upload identity document online, automatically reads the important details from them, and gives a staff member a simple screen to approve or reject the request.

This project was chosen because it covers almost every important concept a beginner needs to learn in full-stack development: a frontend that users interact with, a backend that processes requests and applies rules, a database that stores information permanently, and a security system that makes sure only the right people can access the right data. Because KYC deals with sensitive personal documents such as Aadhaar and PAN cards, it is also a great way to learn why security, validation, and correctness matter so much in real software.

The purpose of this document is to explain, in simple language, how the KYC Verification System is built, what technologies are used, how the different parts of the application talk to each other, and why certain design decisions were made. This document is written for freshers, interns, and students who are new to Spring Boot and want to understand how a real, working application is structured from start to finish.

The main benefits of building this system are, it reduces manual paperwork, it lowers the chance of a document being missed or misread, it gives staff faster visibility into a customer's verification status, and it teaches the reader how a modern three-layer web application is designed, secured, and connected to a database.

2. Objectives

Every project is built to achieve certain goals. For this KYC Verification System, the goals can be grouped into three simple categories: business goals (what the bank gets out of it), technical goals (how the software should be built), and learning goals (what a fresher learns by building it).

2.1 Business Objectives

- Allow the bank to verify a new customer's identity digitally instead of relying on physical paperwork.
- Reduce the time it takes to onboard a new customer.
- Reduce the risk of accepting incomplete or incorrect identity documents.
- Give compliance staff a simple way to review, approve, or reject KYC requests.

2.2 Technical Objectives

- Build a clean, layered backend using Spring Boot, so the code is easy to understand and change.
- Use MySQL to store customer, document, and request data safely.
- Secure the application using Spring Security and JWT so only logged-in staff can review requests.
- Build a REST API so the React frontend and the Spring Boot backend can communicate using simple HTTP requests.
- Automatically extract details such as name, date of birth, and document number from uploaded ID documents using OCR.

2.3 Learning Objectives

- Understand how a frontend (React) and a backend (Spring Boot) work together in a real application.

- Learn how Controller, Service, and Repository layers work together in Spring Boot.
- Learn how login and security work using JWT (JSON Web Token).
- Learn how a relational database is designed using tables and relationships.
- Learn how file upload and basic OCR (reading text from an image) works in a Spring Boot application.
- Get comfortable with common design patterns used in almost every Spring Boot project.

3. Scope

3.1 In Scope

- Login and authentication for staff using JWT.
- Customer registration and submission of KYC documents (Aadhaar, PAN, Passport, photo).
- Automatic extraction of document details using OCR.
- A staff dashboard to review, approve, or reject KYC requests.
- Status tracking of every KYC request (Pending, Under Review, Verified, Rejected).
- Role-based access, for example Admin and Verifier users, have different permissions.

3.2 Out of Scope

- With real integration to government databases such as UIDAI (Aadhaar) or NSDL (PAN), no real government records are contacted.
- Facial recognition or live selfie matching against the ID photo.
- Mobile banking application (this project is a web application only).

3.3 Future Scope

- Integrating with real government verification of APIs for Aadhaar and PAN.
- Adding facial recognition and liveness detection.
- Adding email or SMS notifications for every status change.
- Deploying the application on the cloud so it can be accessed from anywhere.
- Adding Docker so the application can be started with a single command.

4. Technology Stack Used

Layer	Technology	What it is used for
Frontend	React.js	Builds the pages the customer and staff use
Backend	Spring Boot	Handles requests and applies KYC business rules
Database	MySQL	Stores customer, document, and request data permanently

Layer	Technology	What it is used for
ORM	Spring Data JPA / Hibernate	Connects Java code to database tables
Security	Spring Security	Protects the application from unauthorized access
Authentication	JWT	Confirms who the logged-in staff member is
Version Control	Git	Keeps track of code changes over time

4.1 React.js

React.js is a JavaScript library used to build the user interface, the part of the application the user sees, like the document upload page, the login screen, and the staff review dashboard. React lets developers build the page using small, reusable pieces called “components”. For example, an upload box, a status badge, or a review table row can each be a separate component that can be reused wherever needed. React was chosen because it is one of the most popular frontend tools in the industry, and it updates the page quickly without reloading everything every time something changes.

4.2 Spring Boot

Spring Boot is a Java framework used to build the backend of the application, the part that receives requests from React, processes them, and sends back a response. Spring Boot is popular because it removes a lot of complicated setups that older Java frameworks require, and it comes with many built-in features such as security, database connection, and error handling. Spring Boot lets a developer focus on writing the KYC business logic instead of spending time on configuration.

4.3 MySQL

MySQL is a relational database, which means it stores data in the form of tables with rows and columns, like an Excel sheet, but much more powerful. MySQL was chosen because KYC data, customers, their uploaded documents, and their verification requests, is naturally related to each other (for example, one customer can submit many documents), and a relational database is the best tool for storing this kind of connected data safely and consistently.

4.4 JWT (JSON Web Token)

JWT stands for JSON Web Token. It is a small, secure piece of text that is created when a staff member logs in successfully. This token proves who the user is without the backend needing to remember the user's login session. The frontend stores this token and sends it along with every future request, so the backend can quickly check “who is making this request?” Without asking for the username and password repeatedly.

4.5 Spring Security

Spring Security is the part of Spring Boot responsible for protecting the application. It checks whether a user is logged in, verifies their JWT token on every request, and decides whether that user is allowed to access a particular feature. Without Spring Security, anyone could call backend APIs directly and view a customer's personal documents without permission.

4.6 Hibernate / Spring Data JPA

Hibernate is a tool that automatically converts Java objects into database rows, and database rows back into Java objects. Normally, a developer would have to write SQL queries by hand to save or fetch data. With Hibernate (used through Spring Data JPA), a developer can simply call a Java method like `save(kycRequest)`

or `findById(id)` and Hibernate takes care of generating and running the correct SQL behind the scenes. This saves a lot of time and reduces mistakes.

4.7 Tesseract OCR

Tesseract is a free, open-source OCR (Optical Character Recognition) engine that can read printed text from an image. When a customer uploads a photo of their Aadhaar or PAN card, Tesseract scans the image and pulls out the text it can find, such as the name, date of birth, and document number, so the staff member does not have to type these details in by hand. This project uses it as a simple, beginner-friendly starting point.

4.8 Git

Git is a version control tool that keeps track of every change made to the project's code over time.

5. High Level Architecture

Before looking at the small details, it helps to first understand the big picture of how the KYC Verification System works. Figure 1 shows the simplest possible view of the system: a user opens the application in a browser, the browser loads the React frontend, the React frontend talks to the Spring Boot backend, and the Spring Boot backend talks to the MySQL database and to the folder where uploaded documents are kept.

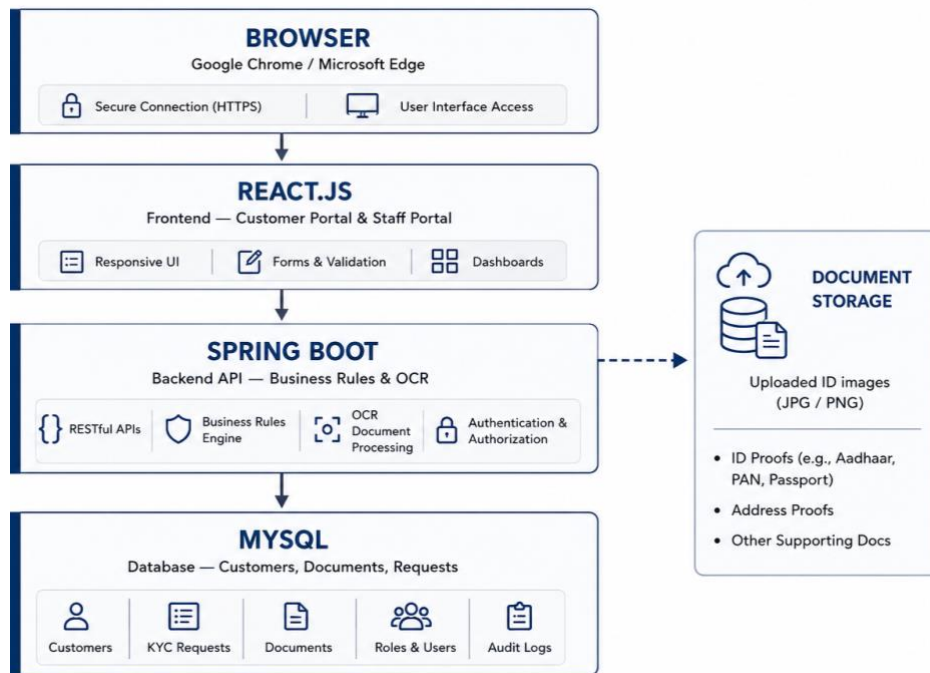


Figure 1: High Level Architecture

Browser: This is where the user actually sees and uses the application, for example, Google Chrome or Microsoft Edge.

React.js: This is the frontend application that runs inside the browser. It shows the customer's document upload form and the staff review dashboard, and it sends the user's actions to the backend.

Spring Boot: This is the backend application. It receives requests from React, checks if the user is allowed to perform the action, applies KYC business rules, runs OCR on uploaded documents, and talks to the database.

MySQL: This is the database where all the information, customers, KYC requests, documents, and verification logs, are permanently stored.

Document Storage: This is where the actual uploaded document images (Aadhaar, PAN, photo) are kept. The database stores a reference (the file path) to each document rather than the image itself.

6. Three-Tier Architecture

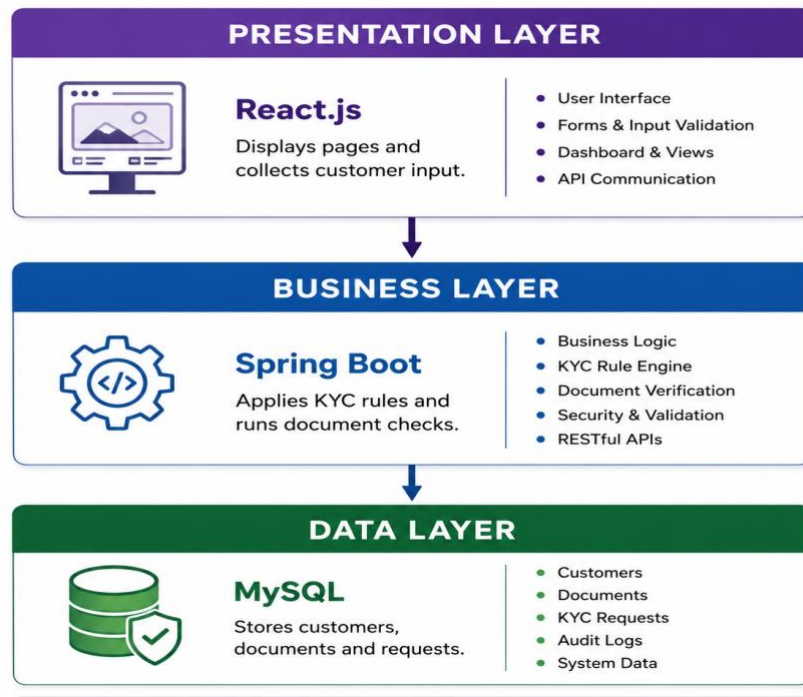


Figure 2: Three-Tier Architecture

6.1 Presentation Layer

The Presentation Layer is what the user actually sees, the document upload page, the request status page, and the staff review screen. In this project, this layer is built using React.js. This layer's only job is to display information nicely and collect input from the user; it does not decide whether a document is valid or not.

6.2 Business Layer

The Business Layer is where all the important decisions and rules are applied, for example, checking whether all the required documents have been uploaded before a request can move to “Under Review”. In this project, this layer is built using Spring Boot, specifically for the Controller and Service classes. This is considered the “brain” of the application.

6.3 Data Layer

The Data Layer is responsible for permanently storing and retrieving data. In this project, this layer is the MySQL database, accessed through Hibernate and Spring Data JPA. This layer does not know anything about business rules; its only job is to save and return data safely.

7. Backend Architecture

The Spring Boot backend itself is further divided into smaller layers so that the code stays clean and organized. Figure 3 shows how a request moves through these layers, and the sections below explain what each layer does in simple terms.

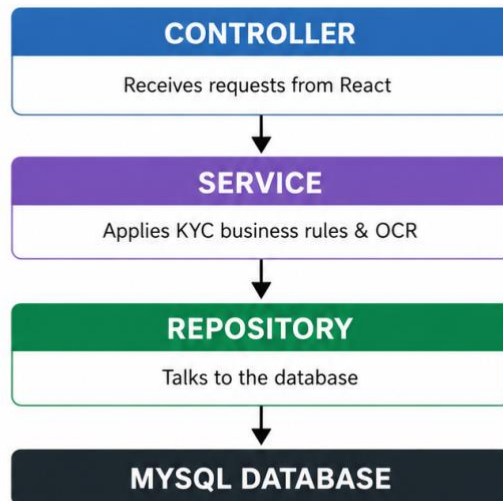


Figure 3: Backend Layer Diagram

7.1 Controller

The Controller is the entry point of the backend. It receives requests coming from React (for example, “upload a document” or “get request status”), and it decides which Service method should handle that request. The Controller does not contain any business logic itself; its only job is to receive the request and send back the response. In this project, KycRequestController and KycDocumentController are examples of controllers.

7.2 Service

The Service layer contains the actual business logic; the rules of how the KYC process should behave. For example, before allowing a request to move to “Under Review”, KycVerificationService checks that all the required documents have been uploaded. The Controller always calls the Service to get work done; it never skips this layer.

7.3 Repository

The Repository layer is responsible for talking to the database. It uses Spring Data JPA, which means a developer can simply write a method name like `findByStatus(status)` and Spring automatically generates the correct database query, no manual SQL writing is required for most operations.

7.4 Entity: Entity is a normal Java class that represents one table in the database. For example, a KycRequest entity class represents the kyc_request table, and each object of that class represents one row (one KYC request) in that table. Hibernate uses these Entity classes to learn how to save and load data.

7.5 DTO (Data Transfer Object)

A DTO is a simple object used only to carry data between the frontend and the backend. Instead of sending the full Entity (which may contain extra internal fields), the backend sends a DTO that contains only the

fields the frontend needs. This keeps the API clean and avoids exposing unnecessary internal details, such as the exact file path where a document is stored on the server.

7.6 Mapper

A Mapper is a small helper class or method that converts an Entity into a DTO, and a DTO back into an Entity. This keeps the conversion logic in one place instead of repeating the same conversion code everywhere it is needed.

7.7 Exception

The Exception package contains custom error classes (for example, `DocumentNotFoundException` or `InvalidDocumentTypeException`) along with a central error handler. Whenever something goes wrong, like a customer trying to upload a file type that is not allowed, this layer makes sure a clear, friendly error message is sent back to React instead of a confusing technical error.

7.8 Configuration

The Configuration package contains settings for the application, for example, which URLs are allowed to call the backend (CORS settings), the maximum size allowed for an uploaded document, or how passwords should be encrypted. These settings are usually written once and rarely changed.

8. Database Design

The database is where all the important information about customers, KYC requests, uploaded documents, and staff decisions is stored. Figure 4 shows the Entity-Relationship (ER) Diagram, which is simply a picture that shows all the tables in the database and how they are connected to each other.

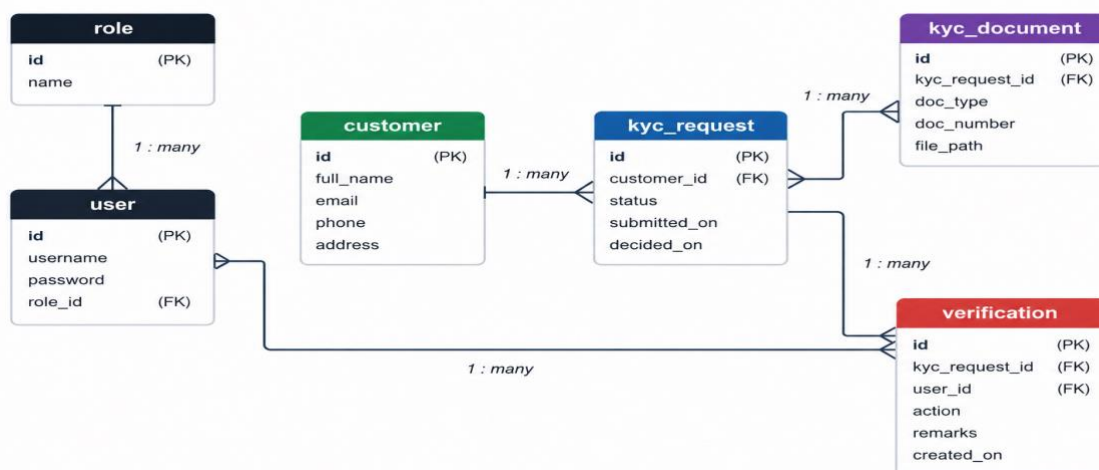


Figure 4: Database ER Diagram

8.1 Table Descriptions

role: Stores the different types of staff users, such as ADMIN or VERIFIER. This decides what a user is allowed to do in the system.

user: Stores the login details (username and password) of staff members who can log in to the system, such as verifiers or admins. Each user is linked to a role.

customer: Stores the personal details of a person applying for KYC verification, such as name, email, phone number, and address.

kyc_request: Represents one KYC application submitted by a customer. Stores its status (Pending, Under Review, Verified, or Rejected) along with when it was submitted and when it was decided.

kyc_document: Stores each document uploaded for a KYC request, the document type (Aadhaar, PAN, Passport), the document number, and the file path where the image is stored.

verification_log: Stores a record of every action taken on a KYC request, who performed it, what action was taken, and any remarks. This acts like an audit trail for the whole verification process.

9. Authentication

Authentication is the process of confirming who a user is, normally by checking a username and password. This section explains how a staff member logs into the KYC Verification System, and how the system remembers that the user is logged in for every request afterward.

9.1 JWT

JWT stands for JSON Web Token. It is a small piece of text created by the backend after a successful login. This token contains information about who the user is, and it is digitally signed so that it cannot be faked or changed by anyone else. The frontend stores this token and attaches it to every future request.

9.2 Login Flow

- The staff member enters a username and password on the React login page and clicks “Login”.
- React sends this information to the backend as a login request.
- Spring Security receives the request and passes it to the Authentication Manager.
- The Authentication Manager checks the username and password against the stored (encrypted) password in the database.
- If the credentials are correct, the backend generates a JWT token and sends it back to React.
- React stores this token (usually in the browser's local storage) so it can be reused for future requests.

9.3 Authorization

Authorization is different from authentication. Authentication confirms who you are; authorization decides what you are allowed to do. For example, a Verifier may be allowed to review a KYC request and add remarks, while only an Admin may be allowed to give the final approval or permanently delete a record. In this project, Spring Security checks the user's role (stored in the JWT token) before allowing access to certain features.

9.4 Protected APIs

A “protected API” is an endpoint that cannot be accessed unless the user provides a valid JWT token. For every request to a protected API, the token is sent in the request header, and Spring Security checks whether the token is valid and whether the user has permission to perform that action. If the token is missing, invalid, or expired, the request is rejected before it ever reaches the Controller or Service layer.

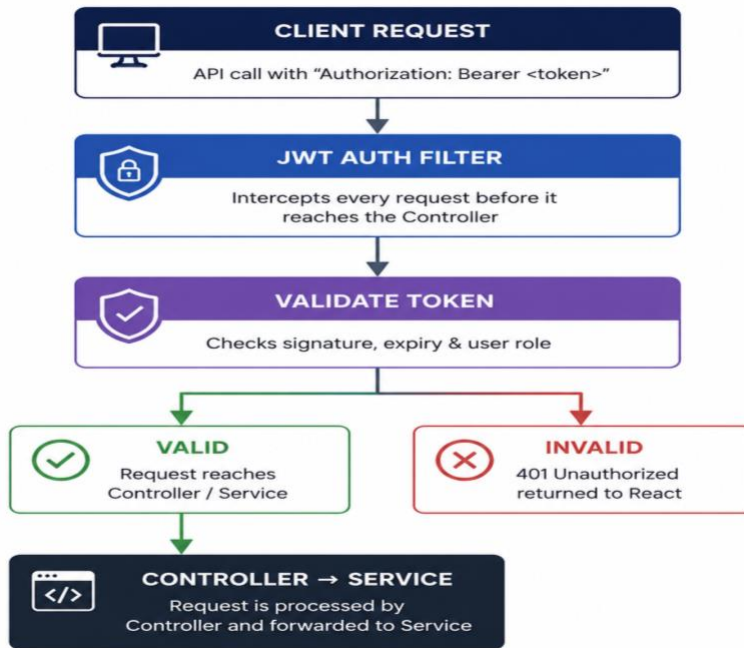


Figure 6: Security Architecture

10. Request Flow

This section explains, step by step, exactly what happens from the moment a customer clicks “Submit” on the document upload form in React, to the moment their KYC request is saved, and a confirmation appears on the screen. Understanding this flow is one of the most important things a fresher can learn, because almost every feature in the application follows this same pattern.

- 1. React: the customer uploads Aadhaar, PAN, and a photo, then click “Submit”.
- 2. Controller: the request reaches KycRequestController, which identifies which method should handle it.
- 3. DTO Validation: the data sent from React is checked to make sure it is complete and correctly formatted (for example, a required document is not missing).
- 4. Service: OCR extracts the document details, and business rules are applied, for example, checking that the same document number has not already been used for another approved request.
- 5. Repository: the Service asks the Repository to save the KYC request and its documents.
- 6. Database: the record is stored in MySQL with a status of PENDING.
- 7. Response: the result travels back up through the same layers and is sent to React as a JSON response, which React then displays to the customer as “Submitted, pending review”

11. Folder Structure

A well-organized folder structure makes it much easier for any developer, especially a fresher joining the project for the first time, to find where a particular piece of code lives. Figure 9 shows the standard Spring Boot package structure used in this project.

Package	What it will contain
controller	Classes that receive API requests from React, such as uploading a document or checking status
service	Classes that contain the KYC business logic
repository	Interfaces that talk to the database
entity	Java classes that represent database tables
DTO	Simple objects used to send/receive data through the API
mapper	Classes that convert Entities to DTOs and back
security	Classes for login, JWT, and access rules
exception	Custom error classes and the global error handler
configuration	Application settings and configuration classes

12. Deployment

The React.js frontend is deployed as static web assets on a web server, while the Spring Boot backend is packaged as an executable JAR and hosted on an application server. MySQL serves as the persistent database for application data, and uploaded KYC documents are stored in a dedicated local storage location. Application configuration, including database connectivity and storage paths, is managed through external configuration files to support flexible deployment.

13. Future Enhancements

Real Government API Integration: Connecting to real UIDAI (Aadhaar) or NSDL (PAN) verification services instead of only extracting text with OCR, so document authenticity can actually be confirmed.

Facial Recognition & Liveness Detection: Comparing a customer's live selfie against the photo on their ID document, and checking that the selfie is a real, live person and not a printed photo.

Redis: A very fast, temporary storage system that can be used to remember frequently requested data, such as a request status, so the database does not need to be queried every single time. Similar to cache.

Cloud Deployment: Instead of running the application on a personal computer, it can be deployed on cloud platforms like AWS or Azure, making it accessible to users anywhere in the world.

Email/SMS Notifications: Automatically notifying a customer whenever their KYC request status changes, so they are always informed without needing to check manually.

14. Conclusion

This Technical Architecture Document presented the design and implementation of the Customer KYC Verification System, developed using React.js, Spring Boot, and MySQL. The application follows a clean three-tier architecture, with the backend organized into Controller, Service, and Repository layers to ensure modularity, maintainability, and scalability. Security is implemented using Spring Security and JWT-based authentication to protect application resources and enforce role-based access control. Overall, the proposed

architecture provides a robust, secure, and extensible foundation for an efficient KYC verification process, while allowing future enhancements such as AI-driven document verification, OCR integration, and cloud-based deployment.